Experiment No. 9

Title

Map Coloring Using Constraint Satisfaction Problem (CSP)

Aim

To develop a Python program that solves the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.

Objective

The objective of this experiment is to understand Constraint Satisfaction Problems by assigning colors to map regions such that no two adjacent regions have the same color.

Algorithm

Step 1: Define the map and neighboring regions.
Step 2: Define the available colors.
Step 3: Select an uncolored region.
Step 4: Assign a valid color to the region.
Step 5: If a conflict occurs, backtrack and choose another color.
Step 6: Repeat until all regions are colored.
Step 7: Display the final color assignment.

Flowchart

```
      Start
        |
        ▼
   Read Map and Colors
        |
        ▼
  Select Uncolored Region
        |
        ▼
   Assign Valid Color
        |
        ▼
```

```
      Conflict Exists?
        |        |
       Yes       No
        |        |
        ▼        ▼
    Backtrack   Next Region
        |        |
        └────────◀────────┘
             |
             ▼
     All Regions Colored?
        |        |
        No       Yes
        |        |
        └────────◀────────┘
             |
             ▼
       Display Result
             |
             ▼
           Stop
```

Python Program

```
states = ['A', 'B', 'C', 'D']


neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
```

```
colors = ['Red', 'Green', 'Blue']

assignment = {}

def is_safe(state, color):
    for neighbor in neighbors[state]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve(index):
    if index == len(states):
        return True

    state = states[index]

    for color in colors:
        if is_safe(state, color):
            assignment[state] = color

            if solve(index + 1):
                return True

            del assignment[state]

    return False

solve(0)

print("Map Coloring Solution:")
```

```
for state in states:
    print(state, "->", assignment[state])
```

Sample Output

Map Coloring Solution:

A -> Red

B -> Green

C -> Blue

D -> Red

Result

The map was successfully colored using the Constraint Satisfaction Problem (CSP) without assigning the same color to adjacent regions.

Conclusion

Thus, the Map Coloring Problem was successfully solved using the Constraint Satisfaction Problem (CSP) approach, ensuring that adjacent regions were assigned different colors through backtracking.